КЛИЕНТ, ПРИЛОЖЕНИЕ И POSTGRESQL

Сервис обработки чисел

Сервис принимает целое число n от 0 до N. Если проверки пройдены, он возвращает n + 1. При повторном обращении с тем же числом или при поступлении числа на единицу меньше уже обработанного возвращается ошибка, которая также попадает в журнал.

Порядок принятия решения

Сначала приложение проверяет тип и диапазон входа. Затем начинает транзакцию и получает общую блокировку. Под ней выполняются две проверки: встречалось ли n в истории и было ли успешно обработано n + 1. Проверка повтора имеет приоритет.

При успехе сохраняется входное число n. Для каждого корректного числа, включая бизнес-отказ, записывается исход обращения. После COMMIT приложение отправляет ответ. При технической ошибке изменения отменяются через ROLLBACK.

Четыре запроса подряд

5: ответ 6. Число принято впервые.
5: ошибка повтора. Предыдущее обращение уже есть в истории.
4: ошибка. Число 5 уже успешно обработано.
6: ответ 7. Ни одно из запрещающих условий не выполнено.

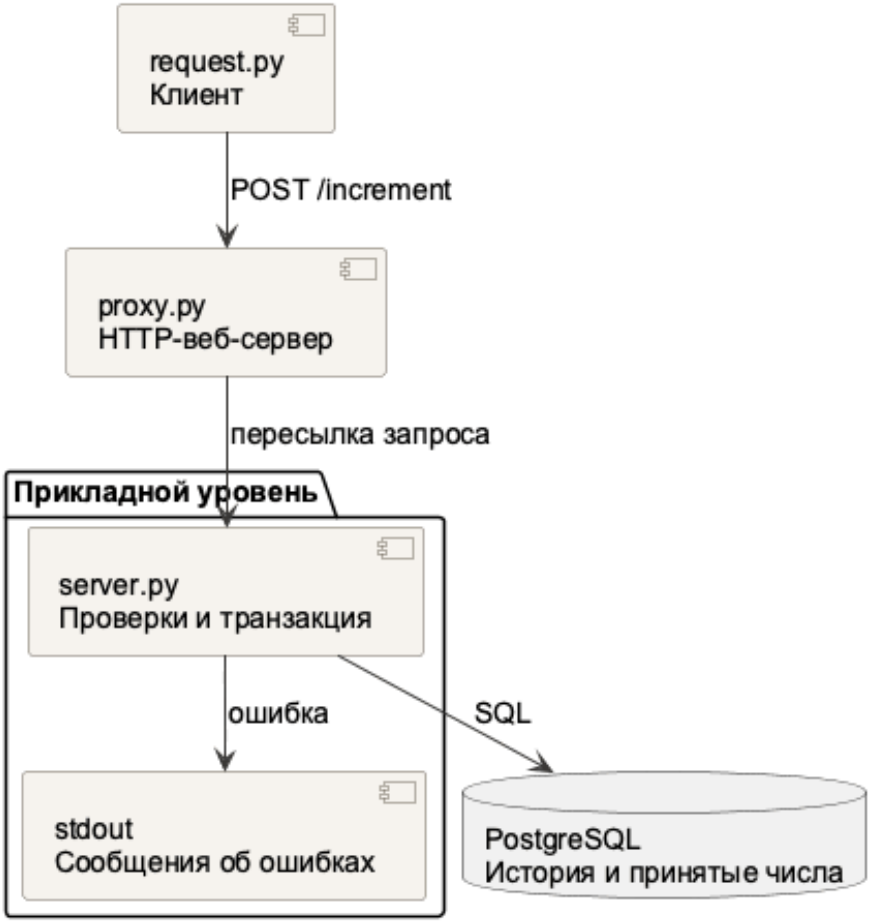
Почему история разделена

Таблица request_history хранит все завершённые обращения с корректными числами, а processed_numbers - только успешно принятые входы. Это различает «число поступало» и «число обработано». После отказа для 4 повторная отправка 4 считается повтором, но наличие отклонённого 4 не запрещает обработку 3.

Условие: Achievement #2, слайд 17 второй лекции. Формат HTTP, таблицы и трактовка истории являются решениями реализации. Основной исходник: **number_service/server.py**.

ДИАГРАММА КОМПОНЕНТОВ

Устройство системы



Роли программ

request.py формирует запрос и показывает ответ. **proxy.py** работает отдельным веб-сервером и пересылает HTTP. **server.py** проверяет число, управляет транзакцией и пишет ошибки в stdout. PostgreSQL сохраняет историю между запусками.

Представление, прикладная логика и данные образуют три логических уровня. Веб-сервер и приложение входят в прикладную часть, хотя выполняются разными процессами.

Формат обращения

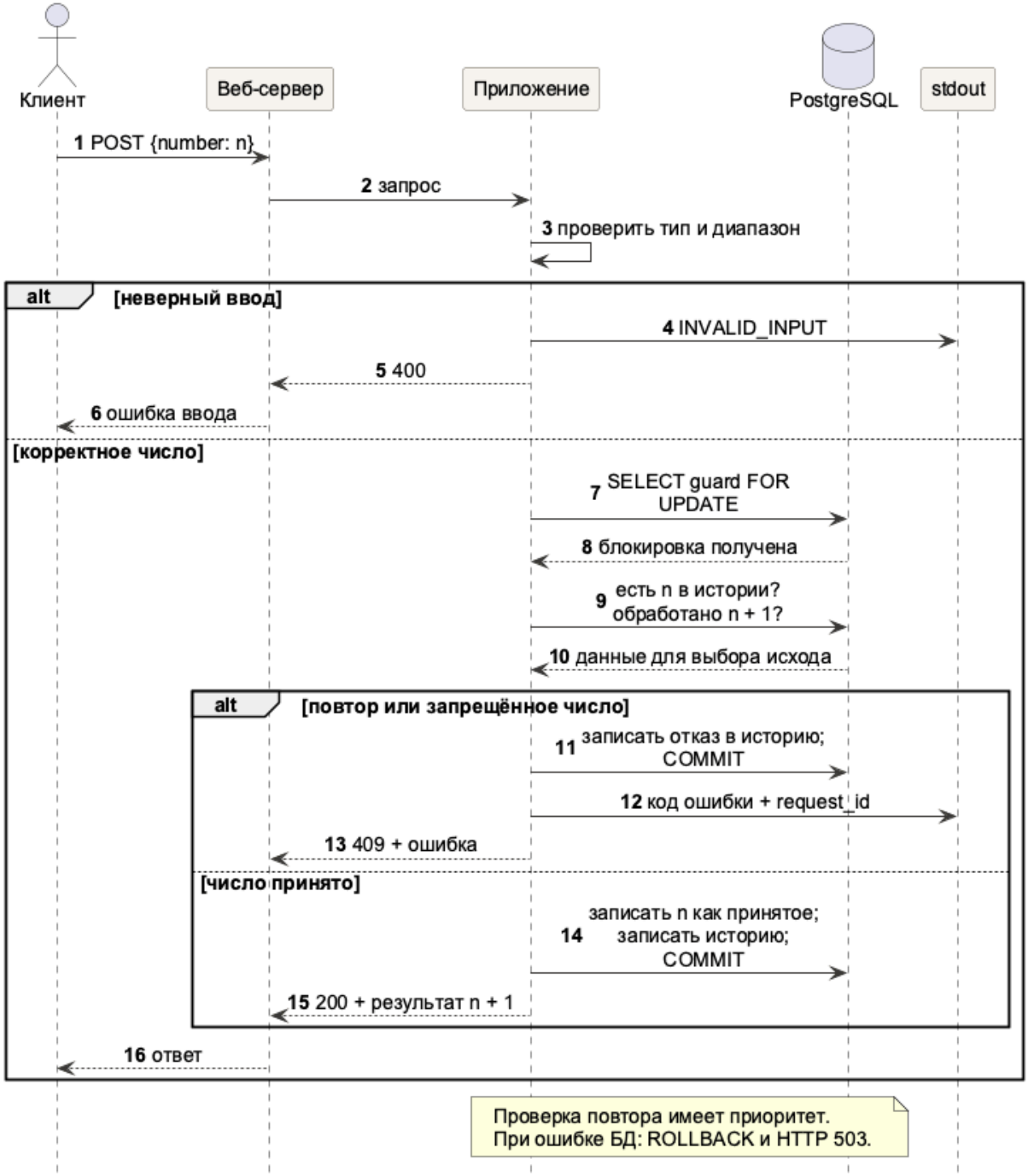
```
POST /increment
Content-Type: application/json

{"number": 5}
```

Успех: HTTP 200 и result=6. Бизнес-отказ: 409 и код ошибки. Неверный ввод: 400. Недоступность базы: 503. Если прокси не может обратиться к приложению: 502. В ответ добавляется request_id для поиска записи в журнале.

ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТЕЙ

Обработка одного запроса



Стрелки читаются сверху вниз. Блок alt разделяет исходы. Проверки и запись результата выполняются внутри одной транзакции; блокировка освобождается при её завершении. Исходники обеих схем объединены в number_service/architecture.puml.

КОД И ПРОВЕРКА

Согласованность и запуск

Зачем нужна общая блокировка

Два запроса могли бы одновременно увидеть отсутствие числа и оба принять его. Функция evaluate_number сначала блокирует одну строку processing_guard. Второй запрос ждёт, затем читает уже обновлённую историю. Это работает и между разными процессами приложения.

```
cur.execute('SELECT id FROM processing_guard WHERE id=1 FOR UPDATE')
if cur.fetchone() is None:
    raise RuntimeError('Missing processing guard')
```

Используется READ COMMITTED. Проверки истории выполняются отдельными SQL-запросами после получения блокировки. Цена простоты - последовательная обработка критической секции даже для разных чисел.

Результаты проверки

После обновления имён файлов прошли все **10 интеграционных тестов** с настоящей PostgreSQL и двумя процессами приложения. Среди 20 одновременных одинаковых запросов получен один успех и 19 ошибок повтора. Проверены оба кода ошибок в stdout и их отсутствие в stderr, неверный ввод, гонки, сохранение истории, откат записи и недоступность сервисов.

Запуск

```
cd number_service
python3 -m pip install -r requirements.txt
python3 start.py --pg-bin /path/to/postgresql/bin
# In another terminal:
python3 request.py 5
```

Нужны Python 3.12+, pg8000 и установленный PostgreSQL. Команда start.py поднимает собственную локальную базу, приложение и прокси. Ctrl-C завершает процессы; данные остаются в .runtime. По умолчанию используются порты 8080, 8081 и 54329.

Повторить тесты: **python3 verify.py --pg-bin /path/to/postgresql/bin**. Они создают временную базу. Результат текущего прогона находится в verification.json. Сервис рассчитан на локальную демонстрацию, без авторизации и TLS.

Технический источник: PostgreSQL 17: Explicit Locking.